



TAREA 2

Diseño Web I

Fecha

Lunes 27 de abril de 2026

Estudiante:

Laura Pérez Ulloa

Profesor:

Álvaro Cordero Peña

Fase A

Landing Page

¿Qué es un landing page?

Una landing page es una página web diseñada para que el usuario realice una acción concreta, como registrarse, comprar o descargar contenido. A diferencia de otras páginas, su estructura es simple, directa y enfocada, evitando elementos innecesarios que puedan distraer.

Generalmente, las personas llegan a una landing page desde anuncios, redes sociales o campañas digitales. Por esta razón, su contenido mantiene coherencia con ese mensaje inicial, reforzando la propuesta y facilitando que el usuario entienda rápidamente qué se espera de él.

Características:

- Tiene un objetivo único (conversión)
- Incluye un llamado a la acción claro (CTA)
- Contenido directo y fácil de entender
- Diseño simple y sin distracciones
- Uso de imágenes o elementos visuales atractivos
- Está alineada con una campaña o anuncio específico
- Busca generar acción inmediata del usuario

Estas características permiten que la experiencia sea clara y efectiva, evitando confusión y guiando al usuario de manera natural dentro de la página. Cada elemento cumple una función específica dentro del recorrido.

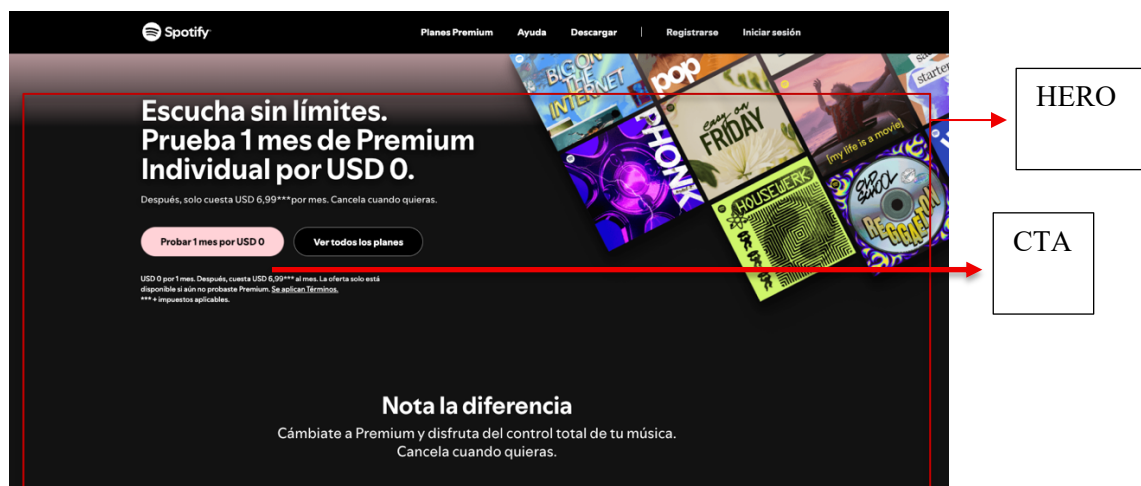
Además, la simplicidad en el diseño y el enfoque en un solo objetivo ayudan a aumentar la probabilidad de que el usuario complete la acción, ya que se reducen las distracciones y se facilita la toma de decisión.

¿Cuál es su objetivo principal?

El objetivo principal de una landing page es convertir visitas en resultados medibles, como registros, compras o descargas. Para lograrlo, se enfoca en presentar la información de forma clara y persuasiva, facilitando el proceso para el usuario.

Todos los elementos dentro de la página, como el diseño, el contenido y los botones de acción trabajan en conjunto para guiar al usuario de manera eficiente, reduciendo fricciones y aumentando las probabilidades de conversión.

Análisis de ejemplo



Organización del sitio

Este landing page tiene un header con menú con tres llamados a la acción (Descargar, Registrarse e iniciar sesión), seguido de una sección HERO con el CTA, beneficios y planes.

Interacción web

La interacción web se refiere a cómo responde una página cuando el usuario realiza acciones como hacer clic, desplazarse o pasar el cursor sobre un elemento. Estas respuestas ayudan a que la experiencia sea más intuitiva, ya que el usuario recibe señales visuales de lo que está ocurriendo.

Además, una buena interacción mejora la usabilidad del sitio, guiando al usuario de manera natural. No se trata solo de que algo se vea bonito, sino de que cada cambio tenga un propósito claro y facilite la navegación.

1. *Microinteracciones (hover)*

Las microinteracciones son pequeños cambios visuales que ocurren cuando el usuario interactúa con un elemento, como pasar el mouse sobre un botón. Estas acciones suelen ser rápidas y sutiles, pero ayudan a comunicar que el elemento es interactivo.

Este tipo de interacción es clave para mejorar la experiencia, ya que le da retroalimentación inmediata al usuario. Por ejemplo, un botón que cambia de color indica claramente que se puede hacer clic.

Ejemplo:

```
CSS
button {
  background-color: #222;
  color: white;
  padding: 10px 20px;
  border: none;
  cursor: pointer;
}

button:hover {
  background-color: #555;
}
```

Hover con escala (efecto “botón vivo”)

Hace que el botón crezca un poco cuando pasás el mouse.

Ejemplo:

```
CSS
button {
  padding: 10px 20px;
  background: #222;
  color: white;
  border: none;
  transition: transform 0.2s ease;
}

button:hover {
  transform: scale(1.05);
}
```

Sombra al hacer hover (elevación)

Simula que el elemento “se levanta”.

Ejemplo:

```
CSS
.card {
  padding: 20px;
  background: white;
  transition: box-shadow 0.3s ease;
}

.card:hover {
  box-shadow: 0 10px 20px rgba(0,0,0,0.2);
}
```

Subrayado animado en links

Se ve mucho más moderno que el subrayado normal.

Ejemplo:

```
css
a {
  position: relative;
  text-decoration: none;
  color: #222;
}
a:after {
  content: "";
  position: absolute;
  left: 0;
  bottom: -3px;
  width: 0;
  height: 2px;
  background: #222;
  transition: width 0.3s;
}
a:hover:after {
  width: 100%;
}
```

¿Qué otras propiedades pueden cambiar?

En una interfaz web, las propiedades que se modifican deben ayudar a que el usuario entienda mejor qué puede hacer en la página. Por ejemplo, cambiar el color de un botón al hacer hover indica que es interactivo, mientras que modificar el tamaño (scale) puede reforzar que ese elemento es importante o clicable.

También se pueden usar cambios en la sombra para dar sensación de profundidad, en la opacidad para mostrar estados activos o inactivos, y en la posición (translate) para simular movimiento. Estos cambios no son solo visuales, sino que comunican estados y guían al usuario de forma intuitiva.

¿Cómo mejorar la experiencia visual?

Para mejorar la experiencia visual es importante que las interacciones sean claras, suaves y coherentes. El uso de transiciones permite que los cambios no sean bruscos, lo que hace que la navegación se sienta más fluida y agradable para el usuario. Además, mantener un estilo consistente en todo el sitio ayuda a que la experiencia sea más intuitiva.

También es clave no sobrecargar la página con demasiados efectos. Las microinteracciones deben tener un propósito, como guiar al usuario o indicar que un elemento es clicable. Cuando se usan correctamente, mejoran la usabilidad, generan una mejor percepción del sitio y facilitan que el usuario complete acciones dentro de la página.

2. Transiciones CSS

Las transiciones en CSS permiten que los cambios en las propiedades de un elemento ocurran de forma gradual en lugar de instantánea. Esto significa que cuando un elemento cambia, por ejemplo de color o tamaño, el cambio se ve suave y progresivo, mejorando la experiencia visual del usuario.

Se utilizan principalmente en interacciones como hover o focus, ya que hacen que la interfaz se sienta más fluida y natural. Sin transiciones, los cambios serían bruscos, lo que puede hacer que el sitio se vea menos profesional o difícil de usar.

Propiedad transition

La propiedad transition es la que se utiliza para definir cómo se va a realizar el cambio entre un estado inicial y uno final. Permite indicar qué propiedad se va a animar, cuánto tiempo durará el cambio y qué tipo de animación se aplicará.

Por ejemplo, se puede usar para suavizar el cambio de color de un botón cuando el usuario pasa el mouse sobre él. Esto hace que la interacción sea más agradable y fácil de percibir.

Ejemplo:

```
CSS
button {
  background-color: #222;
  color: white;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #555;
}
```


Duración y tipos de animación

La duración define cuánto tiempo tarda en completarse la transición. Se expresa en segundos (s) o milisegundos (ms). Una duración corta hace que el cambio sea rápido, mientras que una más larga lo hace más lento y visible.

El tipo de animación (timing function) controla la velocidad del cambio durante ese tiempo. Algunos de los más usados son:

- *ease: comienza lento, acelera y termina lento (más natural)*
- *linear: velocidad constante*
- *ease-in: empieza lento*
- *ease-out: termina lento*
- *ease-in-out: combinación de inicio y final suave*

Ejemplo:

```
CSS
.caja {
  width: 100px;
  height: 100px;
  background: #222;
  transition: transform 0.5s ease-in-out;
}

.caja:hover {
  transform: scale(1.2);
}
```

3. Animaciones con @keyframes

Las animaciones con @keyframes en CSS permiten crear movimientos o cambios visuales más complejos dentro de una página web. A diferencia de las transiciones, no dependen únicamente de una interacción del usuario, sino que pueden ejecutarse automáticamente o repetirse en el tiempo.

Este tipo de animación se utiliza para dar dinamismo a los elementos, como hacer que aparezcan, se desplacen o cambien de forma progresiva. Bien usadas, ayudan a dirigir la atención del usuario y a mejorar la experiencia visual sin necesidad de usar JavaScript.

¿Cómo funcionan?

La regla `@keyframes` define los distintos estados de una animación a lo largo del tiempo. Se establecen puntos clave (por ejemplo, inicio y final) donde se indican los cambios que tendrá el elemento, como posición, tamaño u opacidad.

Luego, esa animación se aplica a un elemento usando la propiedad `animation`, donde se especifica la duración, el tipo de movimiento y si se repite o no.

Ejemplo:

```
CSS
@keyframes aparecer {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

.caja {
  width: 100px;
  height: 100px;
  background: #222;
  animation: aparecer 1s ease-in;
}
```

¿Para qué sirven en experiencia de usuario?

Las animaciones ayudan a:

- Llamar la atención sobre elementos importantes.
- Indicar que algo está pasando (carga, cambio, feedback).
- Hacer la interfaz más dinámica y agradable.

Uso de porcentajes (0%, 50%, 100%)

En las animaciones con `@keyframes`, los porcentajes indican en qué momento de la animación ocurre cada cambio. El 0% representa el inicio, el 100% el final, y cualquier valor intermedio (como 50%) define estados durante el proceso.

Esto permite crear animaciones más complejas, ya que no solo se define un inicio y un final, sino también cómo se comporta el elemento a lo largo del tiempo. Por ejemplo, se puede hacer que un objeto se mueva, se detenga a la mitad y luego continúe, dando una sensación más natural y dinámica.

Ejemplo:

```
CSS
@keyframes mover {
  0% {
    transform: translateX(0);
  }
  50% {
    transform: translateX(50px);
  }
  100% {
    transform: translateX(100px);
  }
}

.caja {
  animation: mover 2s ease-in-out;
}
```

Propiedades animables

Las propiedades animables son aquellas que pueden cambiar gradualmente en el tiempo dentro de una animación. No todas las propiedades de CSS se pueden animar, solo aquellas que permiten una transición progresiva entre valores.

Algunas de las más utilizadas son:

- opacity (transparencia)
- transform (scale, translate, rotate)
- background-color
- width y height
- box-shadow

Estas propiedades son clave porque permiten crear efectos visuales suaves sin afectar negativamente el rendimiento. En especial, transform y opacity son las más recomendadas, ya que son más eficientes y fluidas.

Ejemplo:

```
CSS
@keyframes efecto {
  0% {
    opacity: 0;
    transform: scale(0.8);
  }
  100% {
    opacity: 1;
    transform: scale(1);
  }
}

.elemento {
  animation: efecto 0.5s ease-out;
}
```

4. Scroll suave

El scroll suave es una técnica que permite que el desplazamiento dentro de una página web se realice de forma gradual en lugar de saltar directamente de una sección a otra. Esto mejora la experiencia del usuario, ya que el movimiento se percibe más natural y facilita entender hacia dónde se está navegando.

Se utiliza principalmente cuando hay enlaces internos en la página, como un menú que lleva a distintas secciones. En lugar de un cambio brusco, el scroll suave guía visualmente al usuario, ayudando a mantener el contexto dentro del sitio.

¿Cómo se implementa?

El scroll suave se puede lograr fácilmente utilizando la propiedad scroll-behavior en CSS. Esta propiedad se aplica generalmente al elemento html y controla cómo se comporta el desplazamiento al hacer clic en enlaces internos.

Cuando se establece como smooth, el navegador realiza la transición de desplazamiento de manera progresiva sin necesidad de usar JavaScript.

Ejemplo:

CSS

```
html {  
  scroll-behavior: smooth;  
}
```

HTML

```
<a href="#seccion2">Ir a sección 2</a>  
<section id="seccion2">  
  <h2>Sección 2</h2>  
</section>
```

Navegación interna con anclas

La navegación interna con anclas permite moverse dentro de una misma página web hacia secciones específicas. Se logra utilizando enlaces (<a>) que apuntan a un elemento con un identificador (id), lo que hace que el navegador lleve al usuario directamente a ese punto.

Este tipo de navegación es muy útil en páginas largas o tipo landing, donde el contenido está dividido en secciones. En lugar de hacer scroll manual, el usuario puede ir directamente a la información que necesita con un solo clic.

Ejemplo:

HTML

```
<a href="#contacto">Ir a contacto</a>  
<section id="contacto">  
  <h2>Contacto</h2>  
</section>
```

Mejora de experiencia de usuario

La navegación con anclas mejora la experiencia del usuario porque hace que la interacción sea más rápida y directa. El usuario no tiene que buscar la información desplazándose manualmente, sino que puede acceder a ella de forma inmediata.

Además, cuando se combina con scroll suave, la experiencia es aún más clara, ya que el movimiento ayuda a mantener el contexto dentro de la página. Esto hace que la navegación se sienta más intuitiva, organizada y profesional.

5. Efecto Parallax

El efecto parallax es una técnica visual en la que diferentes elementos de una página se mueven a distintas velocidades al hacer scroll. Generalmente, el fondo se desplaza más lento que el contenido del frente, creando una sensación de profundidad y dinamismo.

Este efecto se utiliza mucho en páginas tipo landing para hacer la experiencia más atractiva y visualmente impactante. Bien aplicado, ayuda a guiar la atención del usuario y a darle más vida al contenido sin necesidad de interacciones complejas.

Limitaciones del parallax con CSS

Cuando se implementa solo con CSS, el parallax tiene varias limitaciones. La forma más común es usando background-attachment: fixed, pero este método no funciona bien en todos los dispositivos, especialmente en móviles, donde muchos navegadores lo desactivan por rendimiento.

Además, el control es bastante limitado, ya que no permite manejar fácilmente diferentes velocidades o animaciones más complejas. Esto hace que el efecto sea más básico y menos flexible en comparación con soluciones más avanzadas.

Ejemplo:

```
CSS
.parallax {
  background-image: url("imagen.jpg");
  height: 400px;
  background-attachment: fixed;
  background-position: center;
  background-size: cover;
}
```

Alternativas más avanzadas

Para lograr efectos parallax más complejos, se suelen utilizar herramientas basadas en JavaScript. Estas permiten controlar con mayor precisión cómo se mueven los elementos, ajustar velocidades y crear animaciones más fluidas.

Algunas opciones más avanzadas incluyen el uso de librerías como GSAP (GreenSock) o ScrollMagic, que permiten sincronizar animaciones con el scroll del usuario. Estas herramientas ofrecen mayor control, mejor rendimiento y compatibilidad en distintos dispositivos, aunque requieren un nivel más alto de conocimiento técnico.

Fase B

Justificación de implementación

Aspecto	Justificación
Qué elementos se implementaron	Microinteracciones (hover), transiciones CSS, animaciones con @keyframes y scroll suave (scroll-behavior).
En qué partes del sitio se incorporaron	Hover en navegación, botones y tarjetas. Transiciones en botones, links y cards. Animación en la sección hero. Scroll suave en navegación interna con anclas.
Por qué se eligieron esos elementos	Son soluciones simples y efectivas que mejoran la interacción sin sobrecargar el diseño. Permiten comunicar acciones y estados de forma clara al usuario.
Cómo contribuyen a la experiencia de usuario	Facilitan la navegación, indican qué elementos son interactivos y brindan retroalimentación visual inmediata, haciendo la experiencia más intuitiva.
Qué mejoras aportan a nivel visual e interactivo	Aportan dinamismo y fluidez, evitan cambios bruscos y mejoran la percepción general del sitio, haciéndolo más moderno y atractivo.

Fase C

Justificación de adaptación responsive

Aspecto	Justificación
Elementos modificados de desktop a tablet y mobile	Se modificó la distribución de las tarjetas, el tamaño de algunos textos, los espacios internos, el tamaño del botón, la organización del header y la altura de la imagen principal. Estos cambios permiten que el sitio se adapte mejor a pantallas medianas y pequeñas.
Elementos que se mantuvieron sin cambios	Se mantuvieron el scroll suave, las transiciones CSS y la animación principal, porque funcionan correctamente en los distintos tamaños de pantalla y no afectan la lectura ni la navegación. También se conservó la línea visual general del sitio para mantener coherencia.
Ajuste de tipografía	Se redujo ligeramente el tamaño de algunos textos en tablet y mobile, como títulos, enlaces de navegación, texto del hero y botones. Esto ayuda a conservar la jerarquía visual sin que los textos se vean demasiado grandes o desproporcionados en pantallas pequeñas.
Adaptación de distribución	En desktop, las tarjetas se muestran en tres columnas. En tablet, se reducen a dos columnas para evitar que el contenido se vea comprimido. En mobile, se organizan en una sola columna para mejorar la lectura y facilitar el recorrido vertical.
Ajuste de contenedor	Se ajustó el ancho del contenedor en mobile y se redujeron algunos paddings para aprovechar mejor el espacio disponible.

Aspecto	Justificación
es y espacios	También se reorganizó el header para que la marca y la navegación queden centradas y ordenadas.
Criterios para mejorar legibilidad e interacción	Se priorizó que el contenido fuera fácil de leer, que los botones siguieran siendo visibles y que las tarjetas no se desbordaran. También se revisó que los efectos de hover no afectaran la experiencia en dispositivos táctiles, donde este comportamiento no siempre se percibe igual.
Resultado de la adaptación	Los cambios responsive permiten que el sitio mantenga una estructura clara en desktop, tablet y mobile. La página conserva su identidad visual, pero se ajusta para ofrecer una experiencia más cómoda, ordenada y funcional en diferentes dispositivos.

Análisis del resultado final

Diferencias entre la versión original y la versión final

La versión original del sitio era fluida, pero no estaba optimizada para distintos tamaños de pantalla. Aunque los elementos se adaptaban en cierta medida, no existía una reorganización clara del contenido.

En la versión final responsive, se realizaron ajustes específicos mediante media queries para mejorar la adaptación en desktop, tablet y mobile. El cambio más evidente es la distribución de las tarjetas, que pasa de tres columnas en desktop, a dos en tablet y una en mobile, logrando una estructura más ordenada y fácil de recorrer.

Mejoras en la experiencia de usuario

La implementación responsive permitió mejorar la legibilidad del contenido en dispositivos pequeños, ya que los textos, espacios y elementos se ajustan de forma adecuada a cada pantalla. Esto facilita que el usuario pueda navegar sin esfuerzo.

Además, el uso de microinteracciones, transiciones y scroll suave aporta fluidez a la experiencia, haciendo que la navegación se sienta más intuitiva y natural. Los botones y enlaces mantienen su visibilidad y facilidad de uso en todos los dispositivos.

Dificultades encontradas

Una de las principales dificultades fue controlar el comportamiento de la imagen principal (hero), ya que generaba problemas de altura y provocaba un scroll innecesario. Esto se resolvió ajustando la altura desde el contenedor en lugar de la imagen.

También fue necesario reorganizar correctamente las tarjetas para cumplir con la distribución por columnas en cada breakpoint. Otro reto fue mantener un equilibrio entre diseño visual e interacción sin sobrecargar el sitio con efectos innecesarios.

Posibles mejoras futuras

Como mejora futura, se podría implementar un menú tipo hamburguesa para dispositivos móviles, lo que optimizaría aún más la navegación en pantallas pequeñas. También se podrían agregar más interacciones o animaciones de forma estratégica.

Además, se podría trabajar en mejorar la accesibilidad del sitio y seguir refinando la tipografía y los espacios para lograr una experiencia aún más clara, coherente y profesional.

Referencias

Mozilla Foundation. (s.f.). *CSS transitions*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/CSS/transition>

Mozilla Foundation. (s.f.). *CSS animations*. MDN Web Docs.
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Animations

Mozilla Foundation. (s.f.). *scroll-behavior*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/CSS/scroll-behavior>

Mozilla Foundation. (s.f.). *transform*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/CSS/transform>

W3Schools. (s.f.). *CSS transitions*.
https://www.w3schools.com/css/css3_transitions.asp

W3Schools. (s.f.). *CSS animations*.
https://www.w3schools.com/css/css3_animations.asp

W3Schools. (s.f.). *HTML links*.
https://www.w3schools.com/html/html_links.asp

Nielsen Norman Group. (s.f.). User experience basics.
<https://www.nngroup.com/articles/definition-user-experience/>